

End-to-end protocol for high-quality quantum approximate optimization algorithm parameters with few shots

Tianyi Hao^{1,*}, Zichang He^{1,*†}, Ruslan Shaydulin^{1,‡}, Jeffrey Larson² and Marco Pistoia¹

¹Global Technology Applied Research, *JPMorganChase*, New York, New York 10017, USA

²Mathematics and Computer Science Division, *Argonne National Laboratory*, Lemont, Illinois 60439, USA



(Received 13 December 2024; accepted 1 August 2025; published 21 August 2025)

The quantum approximate optimization algorithm (QAOA) is a quantum heuristic for combinatorial optimization that has been demonstrated to scale better than state-of-the-art classical solvers for some problems. For a given problem instance, QAOA performance depends crucially on the choice of the parameters. While average-case optimal parameters are available in many cases, meaningful performance gains can be obtained by fine-tuning these parameters for a given instance. This task is especially challenging, however, when the number of circuit executions (shots) is limited. In this work, we develop an end-to-end protocol that combines multiple parameter settings and fine-tuning techniques. We use large-scale numerical experiments to optimize the protocol for the shot-limited setting and observe that optimizers with the simplest internal model (linear) perform best. We implement the optimized pipeline on a trapped-ion processor using up to 32 qubits and 5 QAOA layers, and we demonstrate that the pipeline is robust to small amounts of hardware noise. To the best of our knowledge, these are the largest demonstrations of QAOA parameter fine-tuning on a trapped-ion processor in terms of two-qubit gate count.

DOI: [10.1103/24gg-7p8z](https://doi.org/10.1103/24gg-7p8z)

I. INTRODUCTION

Quantum computing has shown great promise in tackling computational problems that are difficult for classical computers. Among such problems, combinatorial optimization problems are of particular interest due to their ubiquity in fields including finance, logistics, and operations research and due to the existence of quantum algorithms offering speedups [1–5]. The quantum approximate optimization algorithm (QAOA) [6–8] is a prominent quantum heuristic that has been demonstrated to achieve better scaling than state-of-the-art classical solvers for certain combinatorial optimization problems, including maximum 8-satisfiability [9] and low autocorrelation binary sequence [10] problems. QAOA solves an optimization problem by preparing a parameterized quantum state such that upon measuring it, a high-quality solution is obtained with high probability. To apply QAOA to a given problem, QAOA parameters must be set.

The performance of QAOA is highly sensitive to the choice of these parameters, and its parameter optimization has been widely studied in the community [11,12]. For many problem classes, optimal parameters have been derived in the

infinite-size limit and empirically demonstrated to achieve good performance for finite-sized instances [9,13–16]. Even when rigorous theoretical analysis is out of reach, one fixed set of empirically obtained QAOA parameters can work well for most instances [10]. Nonetheless, there is still nontrivial variation in the optimal parameters between instances, which is often amplified by adding weights to the problem. Thus, fine-tuning the average-case or infinite-size parameters for a given instance is necessary to fully exploit the algorithm’s potential [17–19]. Fine-tuning these parameters is challenging, however, especially when the number of circuit executions (shots) is limited, as is often the case with current quantum hardware.

Shots are the fundamental currency of near-term quantum computation. One “shot” represents an execution of a quantum circuit followed by a measurement. Each optimization iteration in QAOA requires hundreds or thousands of shots to minimize the sampling error of an expectation value evaluation [20]. The limitations of near-term quantum devices, such as their scarcity, frequent and time-consuming recalibration, and slow operation time, constrain quantum resource availability and, thus, the total number of shots available for an algorithm run. This constraint is particularly pronounced in atomic platforms such as trapped-ion and neutral atom quantum processors, where measurement time is on the same order of magnitude as gate time [21–23].

The optimization of QAOA parameters presents a significant challenge if the number of shots is limited. This challenge is exacerbated by the fundamental limits that quantum mechanics imposes on the cost of computing gradients of quantum circuits [24]. The high cost of computing the gradient motivates the use of derivative-free optimization (DFO),

*These authors contributed equally to this work.

†Contact author: zichang.he@jpmchase.com

‡Contact author: ruslan.shaydulin@jpmchase.com

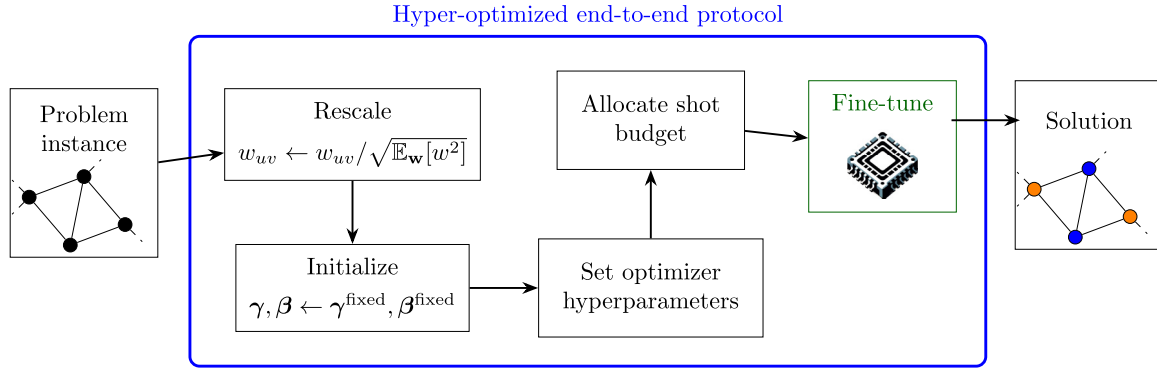


FIG. 1. Overview of our protocol. Given a problem instance, we first follow Ref. [13] to rescale the weights and then set parameters to known good initial points [14,15], based on the parameter concentration property of QAOA [17,41]. We then set the hyperparameters of optimizers and allocate the shot budget. Both hyperparameter choice and the shot budget allocation are informed by extensive numerical investigation detailed in this paper.

which typically either assumes a deterministic objective [25] or requires a high number of shots to converge [26–29]. As a consequence of these challenges and despite the recent progress [30–40], the problem of optimizing parameterized quantum circuits with a small number of shots remains open.

In this work, we propose and implement an end-to-end protocol for obtaining high-quality QAOA parameters with a small number of shots. Our protocol integrates multiple techniques to reduce the cost of parameter optimization, as shown in Fig. 1. These techniques include previously studied ones such as initialization with instance-independent or “fixed” parameters [9,13,14,16,17] and rescaling of weighted problems [13,17], as well as new components to carry out parameter fine-tuning in the shot-frugal setting, including optimizer selection, hyperparameter tuning, and shot budget allocation. We use extensive numerical experiments to optimize all aspects of our pipeline. In doing so, we demonstrate that the optimizer with the simplest internal model is the best option in shot-frugal scenarios. Our protocol performs well without further classical configuring when given a new problem instance.

We demonstrate the effectiveness of our protocol by deploying it on trapped-ion quantum devices applied to QAOA circuits with up to 32 qubits. Our protocol is optimized in noiseless simulation but is robust to small amounts of noise and achieves good performance on hardware. For example, in one instance of 20-qubit 3-regular MaxCut with five QAOA layers, the parameter setting protocol achieves up to 56.61% relative approximation ratio (AR) improvement in noiseless simulation while it holds 46.88% relative AR improvement under hardware noise. We observe that as the circuit size grows, the noise becomes too strong, and the performance of the protocol deteriorates.

II. BACKGROUND

The QAOA [6–8] solves combinatorial optimization problems by preparing a parameterized quantum circuit such that upon measuring it, high-quality solutions are obtained with high probability. The circuit is defined by two operators, problem Hamiltonian H_P and mixer Hamiltonian H_M , a hy-

perparameter p , and an initial state $|\psi_0\rangle$:

$$|\psi(\boldsymbol{\gamma}, \boldsymbol{\beta})\rangle = e^{-i\beta_p H_M} e^{-i\gamma_p H_P}, \dots, e^{-i\beta_1 H_M} e^{-i\gamma_1 H_P} |\psi_0\rangle, \quad (1)$$

where $\boldsymbol{\gamma} = [\gamma_0, \dots, \gamma_p]$ and $\boldsymbol{\beta} = [\beta_0, \dots, \beta_p]$ are free parameters. As the number of layers p approaches infinity, the QAOA circuit with appropriate parameters $|\psi(\boldsymbol{\gamma}, \boldsymbol{\beta})\rangle$ approaches the ground state of the problem Hamiltonian H_P , and the corresponding energy approaches the optimal value of the problem’s objective function. The parameters $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$ are typically obtained by using a classical optimizer that iteratively updates them based on the measurement outcomes:

$$\min_{\boldsymbol{\gamma}, \boldsymbol{\beta}} \langle \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) | H_P | \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) \rangle, \quad (2)$$

where $\langle \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) | H_P | \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) \rangle$ is the expectation value of the energy.

The optimization of variational parameters within the QAOA framework presents a significant challenge. On the one hand, the objective function in practical implementations is inherently stochastic. In each QAOA iteration, the expectation value $\langle \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) | H_P | \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) \rangle$ is estimated and given to the optimizer as the objective by sampling numerous measurement results from the QAOA state:

$$\langle \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) | H_P | \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) \rangle \approx \frac{1}{M} \sum_i^M f(x_i), \quad (3)$$

with $x_i \sim |\psi(\boldsymbol{\gamma}, \boldsymbol{\beta})|^2$,

where $f(x)$ is the associated problem value of a measurement $\mathbf{x}$.

Each measurement represents an execution of the entire circuit and the number of circuit executions used to estimate a state is referred to as the number of shots. The most interesting and promising use cases involve limited shots of the stochastic objective. Based on the central limit theorem [42], with M shots, the standard deviation of estimated energy becomes $\sigma_0/\sqrt{M}$, where

$$\sigma_0 = (\langle \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) | H_P^2 | \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) \rangle - \langle \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) | H_P | \psi(\boldsymbol{\gamma}, \boldsymbol{\beta}) \rangle^2)^{1/2}. \quad (4)$$

This uncertainty in the estimated energy caused by finite sampling becomes significant when the number of shots is small,

such as below 1000, which renders the parameter optimization challenging.

On the other hand, there is limited access to gradients of the QAOA objective, which means practical QAOA experiments have to rely on optimization techniques that do not utilize derivative information, a task that is naturally more complex than gradient-based optimization [25]. In the absence of gradients, quantum computing researchers have turned to DFO techniques as the classical optimization approaches in their QAOA work. DFO approaches can coarsely be categorized into direct-search and model-based methods. Direct-search methods evaluate the objective at a geometric pattern of points around a candidate point. If a better point is observed, the best point is updated; otherwise, the displacement is decreased in the geometric pattern. Model-based methods also evaluate points near a candidate point and use these evaluations to build various local or global models of the objective being optimized.

Applying DFO methods that are designed for deterministic objectives to stochastic objectives often leads to suboptimal performance. Rigorous convergence guarantees for stochastic DFO methods typically demand a substantial number of samples to accurately construct or adjust optimization models [26–28]; applying them to quantum optimization tasks will likely be difficult. There is ongoing research aimed at adapting deterministic DFO methods to better accommodate the inherent noise within stochastic objectives, striving for a balance between robustness and sample efficiency [43].

In this paper, we apply QAOA to weighted maximum cut (MaxCut) and portfolio optimization (PO) problems. We now briefly discuss how QAOA is instantiated to be applied to these problems.

MaxCut Given an undirected graph $G = (V, E)$ with an edge weight w_{uv} associated with each edge $(u, v) \in E$, find $s \in \{-1, 1\}^{|V|}$ that will

$$\text{maximize } f(s) = \sum_{(u,v) \in E} \frac{w_{uv}}{2} (1 - s_u s_v).$$

Mapping spin variables s_i onto the spectrum of Pauli Z matrices, we obtain the Hamiltonian that encodes the MaxCut problem on qubits:

$$H_P = \sum_{(u,v) \in E} \frac{w_{uv}}{2} (\mathbf{I} - Z_u Z_v). \quad (5)$$

We use the Pauli X mixer Hamiltonian when applying QAOA to the MaxCut problem:

$$H_M = \sum_i X_i. \quad (6)$$

The initial state $|\psi_0\rangle$ is set to be the ground state of the mixer Hamiltonian, which for H_M in Eq. (6) is

$$|\psi_0\rangle = |+\rangle^{\otimes n}. \quad (7)$$

Portfolio Optimization Given assets with expected returns $\mu \in \mathbb{R}^n$ and covariance $\Sigma \in \mathbb{R}^{n \times n}$, a risk factor $q \in \mathbb{R}$, and a budget $K \in \mathbb{N}$, find $x \in \{0, 1\}^n$ that will

$$\begin{aligned} &\text{minimize } f(x) = qx^T \Sigma x - \mu^T x \\ &\text{subject to } \mathbf{1}^T x = K. \end{aligned}$$

Mapping binary variables x_i to the Pauli Z matrices as $x_i \rightarrow (\mathbf{I} - Z_i)/2$, we get the Hamiltonian

$$H_P = \frac{1}{2} q \sum_{i < j} w_{ij} Z_i Z_j - \frac{1}{2} \sum_i \left(q \sum_j \Sigma_{ij} - \mu_i \right) Z_i + c \quad (8)$$

encoding PO on qubits, where $c = (1/2) \sum_i (q \sum_j \Sigma_{ij} - \mu_i)$ is a constant. To preserve the Hamming weight of the state, we use an XY mixer with a one-dimensional ring connectivity defined as

$$H_M = \sum_i \sum_{j=i+1} X_i X_j + Y_i Y_j. \quad (9)$$

The initial state is prepared as a Dicke state [44], which is a superposition of all feasible (i.e., Hamming weight K) bitstrings with an equal probability.

Given a solution s or x to the problem, we use AR to quantify the quality of the solution. For MaxCut, it is defined as

$$\text{AR}(s) = \frac{f(s) - f_{\min}}{f_{\max} - f_{\min}}, \quad (10)$$

where $f_{\min}$ and $f_{\max}$ are the minimum and maximum value of $f(s)$, respectively; that is,

$$f_{\min} = \min_s f(s), \quad f_{\max} = \max_s f(s). \quad (11)$$

For PO, we need to take constraints into consideration:

$$\text{AR}(x) = \begin{cases} \frac{f(x) - f_{\max}}{f_{\min} - f_{\max}}, & \sum_i x_i = K, \\ 0, & \sum_i x_i \neq K, \end{cases} \quad (12)$$

where $f_{\min}$ and $f_{\max}$ are

$$f_{\min} = \min_{\sum_i x_i = K} f(x), \quad f_{\max} = \max_{\sum_i x_i = K} f(x). \quad (13)$$

We also use the metric of relative AR improvement, defined as

$$\frac{\text{AR}(x) - \text{AR}_{\text{ini}}}{\text{AR}_{\text{opt}} - \text{AR}_{\text{ini}}}, \quad (14)$$

where AR_{ini} and AR_{opt} are the approximation ratios corresponding to the collection of solutions produced by QAOA circuits with initial parameters $\gamma_{\text{ini}}, \beta_{\text{ini}}$ and optimal parameters $\gamma_{\text{opt}}, \beta_{\text{opt}}$:

$$\begin{aligned} \text{AR}_{\text{ini}} &= \mathbb{E}_{x \sim |\psi(\gamma_{\text{ini}}, \beta_{\text{ini}})\rangle} [\text{AR}(x)], \\ \text{AR}_{\text{opt}} &= \mathbb{E}_{x \sim |\psi(\gamma_{\text{opt}}, \beta_{\text{opt}})\rangle} [\text{AR}(x)]. \end{aligned} \quad (15)$$

From a practical perspective, the optimal parameters refer to the set of parameters we can empirically find that lead to the best objective function value. In our evaluations, we obtain the optimal parameters by performing noiseless optimizations with unlimited shots.

III. RESULTS

We now present our results. We begin by summarizing our protocol in Sec. III A and briefly introducing existing techniques our protocol uses. We proceed by describing the rest of its components, with optimizer selection in Sec. III B, hyperparameter study in Sec. III C, and budget allocation in

Sec. III D. We then present the performance of the protocol on trapped-ion hardware in Sec. III E.

A. End-to-end protocol for QAOA parameter optimization

Figure 1 shows an overview of our protocol. Given a problem instance, we first follow Ref. [13] to rescale the weights. We divide the objective function

$$\sqrt{\frac{1}{|E_2|} \sum_{i,j} w_{ij}^2 + \frac{1}{|E_1|} \sum_i w_i^2}, \quad (16)$$

where $|E_2|$ is the number of quadratic terms in the objective function and $|E_1|$ is the number of first-order terms. This rescaling rule can also be extended for problems with higher-degree terms.

The difficulty of optimizing the parameters in QAOA heavily depends on the initial point selection. It has been shown for several problem settings that the optimized parameters for different problem instances are approximately equal [9,10,13,14,16,17]. Consequently, the averaged optimized parameters from several problem instances serve as a high-quality initial point. Thus, we use the parameters given in Ref. [15] for unweighted MaxCut with 3-regular graphs as our initial points for MaxCut and follow the empirical observation in Ref. [13] to use the averaged optimized parameters for the Sherrington-Kirkpatrick model [14] for PO. The values of γ and β are further rescaled so that they are on the same scale for the optimizer.

The rest of the components concern optimizer selection, hyperparameter tuning, and shot budget allocation, which we describe in detail in the following subsections.

B. Optimizer choice

We conduct an evaluation of various optimization methods under the shot-frugal setting. Specifically, we examine the performance of COBYLA [45], BOBYQA [46], NELDER_MEAD [47], ESCH [48], DIRECT_L [49], CRS2_LM [50], SPSA [51], GSLs [52], and IMFIL [53]. These methods have been considered in the quantum optimization context and are available in optimization packages NLOPT [54], PY-BOBYQA [55], SCIPY [56], PDFO [57], and SCIKIT-QUANT [58]. Some other methods we also tested but either work very similarly to one of the above methods or perform undesirably include UOBYQA [59], NEWUOA [60], LINCOA [61], and SNOBFIT [62]. We compute the energy landscapes of 60 random $p=1$ PO instances and efficiently test the optimization quality of each method.

Some of the tested methods have dozens of hyperparameters that can be adjusted before running; other methods have only one or two. A complete study of the performance of each method as its hyperparameters change is far beyond the scope of this manuscript. Instead, we focus our numerical studies on the allocation of the total budget, an additional hyperparameter they all share that is crucial to our restricted setting. This is particularly important if the optimization method needs a few initial function evaluations before being able to make a prediction since more function evaluations lead to fewer shots per evaluation. In addition, the “initial step size” of

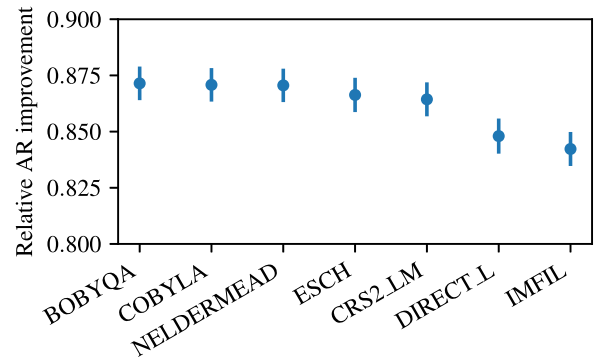


FIG. 2. Performance comparison of common derivative-free optimizers optimizing $p=1$ QAOA circuits for 60 random PO instances. Each optimizer is evaluated under different budget allocation strategies with a total budget of 10 000 shots, and model-based optimizers also have their initial step size grid searched. The best-performed hyperparameter combination is then used to plot this figure. The metric is the mean relative AR improvement with standard error over instances. The optimizers are arranged in descending order of mean AR. BOBYQA, COBYLA, and NELDER_MEAD achieve very comparable performance. The mean relative AR improvement optimized by SPSA is 0.385, which is too low to be included in the figure.

model-based methods, which determines the spread of the initial pattern of points, is a crucial hyperparameter that impacts the performance significantly. We also vary it and choose the best-performing one for these methods in our benchmarking. We test a total of 1460 optimization configurations, and each configuration is evaluated 5 times with different sampling seeds on each of the instances. Please refer to Sec. V B for the details of the efficient performance evaluation.

Figure 2 shows the performance comparison of the tested methods using mean relative AR improvement [Eq. (14)] as the metric. We see that BOBYQA, COBYLA, and NELDER_MEAD perform the best in this setting among all tested methods. We choose COBYLA over NELDER_MEAD since the former is considered an improved version of the latter [45]. Between BOBYQA and COBYLA, we choose the latter, attributing to the fact that a simple model minimizes the number of initial function evaluations and maximizes the number of shots per evaluation. COBYLA assumes a linear model and needs only $2p + 1$ initial function evaluations to build the model for $2p$ parameters in a p -layer QAOA. BOBYQA follows almost the same strategy as COBYLA except that it assumes a quadratic model, which requires up to $(1/2)(2p + 1)(2p + 2)$ initial function evaluations to fully determine the model. We adopt the default setting of $4p + 1$ initial evaluations, which is still almost twice as many as COBYLA requires in the large p limit. In the small p regions, the performance of BOBYQA and COBYLA is comparable (see more details in Appendix). However, the number of shots per evaluation for BOBYQA approaches half of that for COBYLA as we increase the number of QAOA layers (p). The substantial uncertainty due to shotted evaluations outweighs the benefits of fitting a more refined model. Similarly, other methods generally have more complicated assumptions and are thus more demanding in terms of the number of

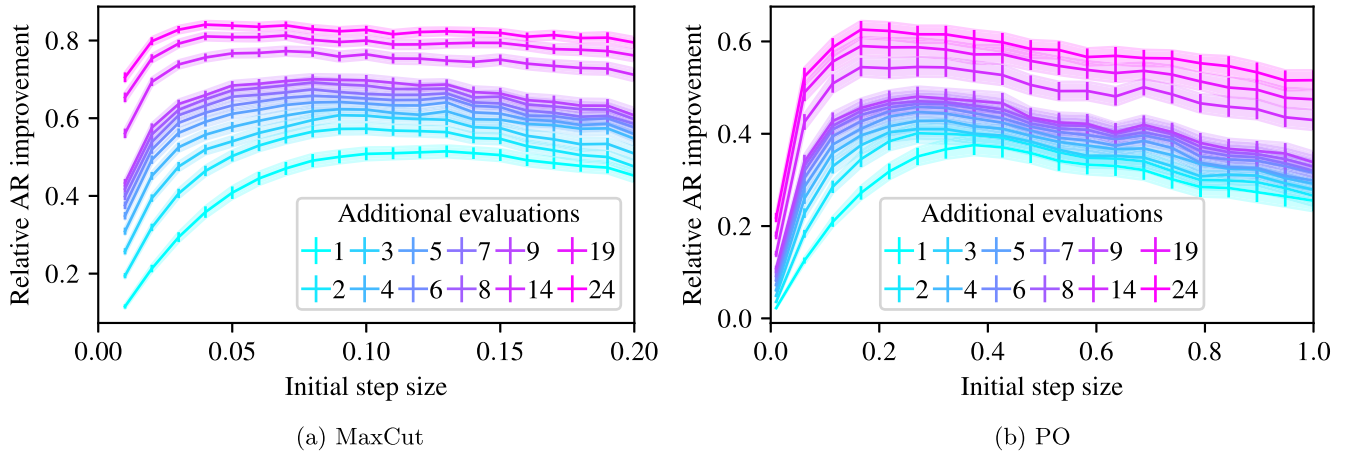


FIG. 3. Mean relative AR improvement (with standard error over instances) of COBYLA on $p = 5$ QAOA instances as a function of initial step size, assuming an infinite shot budget. The label of each line represents the number of function evaluations allowed after the initial evaluations. We observe that MaxCut, having a more accurate initialization strategy, requires a smaller initial step size than does PO. With a given problem and initialization strategy, COBYLA is generally not sensitive to the initial step size.

evaluations. The improved accuracy in the predictions does not compensate for the loss resulting from the significant decrease in the number of shots per evaluation.

C. Hyperparameter selection

We now fix COBYLA as our optimization method and investigate its hyperparameter choice. COBYLA only has one hyperparameter to tune besides the budget allocation strategy, which is the initial step size (“rhobeg”). As previously mentioned, this hyperparameter determines the distance between the initial point and other initial evaluations, which is crucial for establishing an accurate linear model while using few evaluations. We run exact simulations of 100 random 12-qubit MaxCut instances and 60 random 12-qubit PO instances for $p \in \{1, \dots, 5\}$ and plot the optimization results under varying initial step sizes.

In Fig. 3, we plot the mean relative AR improvement with standard error over instances as a function of initial step size, where each curve represents the performance at the given number of function evaluations after the initial evaluations. The performance converges toward the maximum possible relative improvement as we increase the maximum possible evaluations, as expected. We observe that MaxCut, having a more accurate initialization strategy, requires a smaller initial step size than does PO. With a given problem and initialization strategy, we notice that COBYLA is generally not sensitive to the initial step size, even when the number of evaluations is very low. The same trend can be observed across different numbers of QAOA layers (p), which we show in Fig. 12 in Appendix. For subsequent experiments, we use an initial step size of 0.1 for MaxCut and 0.5 for PO.

D. Budget allocation

We now study the budget allocation strategy. We primarily focus on a per-instance total budget of 10 000 shots. This value of total budget was chosen to match the constraints of trapped-ion hardware. Prior QAOA experiments on the device

used in this work used 2000, 5200, and 7800 shots and $p \leq 2$ [21,63]. For comparison, previous hardware demonstration on a superconducting processor used 25 000 shots per evaluation and six evaluations per optimizer iteration [64]. The number of shots per evaluation is inversely proportional to the number of total function evaluations:

$$\text{\#shots/evaluation} = \left\lfloor \frac{\text{shot budget} = 10\,000}{\text{\#evaluations allowed}} \right\rfloor. \quad (17)$$

We perform hyperparameter grid searches on the number of shots per function evaluation versus the maximum number of function evaluations on 1000 random 12-qubit MaxCut instances for $p \in \{2, 3, 4, 5\}$.

Figure 4 shows the mean relative AR improvement with standard error over instances as a function of the maximum number of function evaluations given to the optimizer, starting from the required number of initial function evaluations for building the linear model in COBYLA. We observe that in exchange for more function evaluations, the rapidly reduced

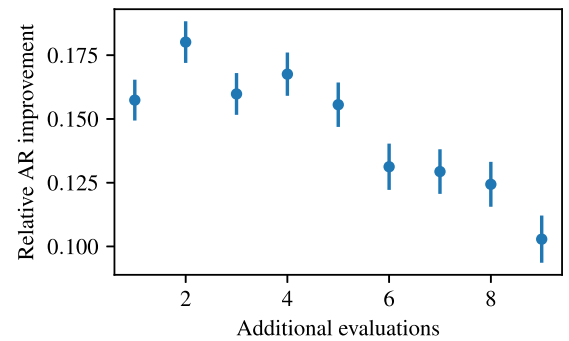


FIG. 4. Mean relative AR improvement (with standard error over instances) of optimizing 1000 $p = 5$ $n = 12$ MaxCut instances as a function of the number of additional evaluations after the first $2p + 1$ initial evaluations. The number of shots per evaluation is divided evenly from a total budget of 10 000 shots. Empirically, we observe that two iterations after the initial evaluations work the best on the average case.

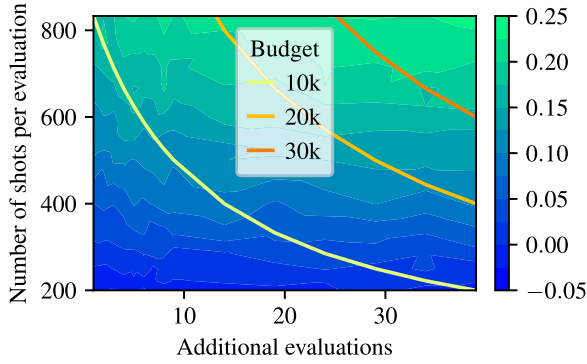


FIG. 5. Contour plot of optimizing 1000 $p = 5$ $n = 12$ MaxCut instances spanned by the number of additional evaluations and the number of shots per evaluation. The color represents mean relative AR improvement and the three lines correspond to a total budget of 10k, 20k, and 30k, respectively.

number of shots per evaluation significantly impacts the measurement accuracy and optimizer behavior. With a good initial point, the optimizer can get close to the optimal point in a few iterations after the initial evaluations. Thus, the best strategy is to maximize the number of shots per evaluation so that the optimizer can rely on the initial evaluations to accurately predict candidate points. Empirically, we observe that two iterations after the initial evaluations work the best on the average case.

Figure 5 shows a contour plot spanned by the number of additional evaluations and the number of shots per evaluation. The color represents mean relative AR improvement and the three lines correspond to a total budget of 10k, 20k, and 30k, respectively. Focusing on the 10k budget line, we make the same observation as in Fig. 4: Smaller number of steps and higher per-evaluation shot budget give the best results. Following the contours, we see that increasing the number of evaluations has little return in contrast to the steady gain in increasing the number of shots per evaluation. If the number of shots is as low as 200, the optimizer will even find lower-quality parameters than the initial ones due to the extreme variance in sampling the objective function values. We expect the budget allocation strategy to be the same for similar budgets, whereas a substantially higher budget can allow more evaluations. We show additional results for $p \in \{2, 3, 4\}$ in Figs. 13 and 14 in Appendix. We can also derive the minimum shot budget requirement from these contour figures, where the budget results in approximately zero improvement in the relative AR.

E. Hardware demonstrations

We demonstrate the effectiveness of our parameter setting protocol with Quantinuum’s noisy emulator of H1-1 (denoted as H1-1E), as well as the H1-1 [65] and H2-1 [66] quantum processors. Each optimization has a total budget of 10k shots across all evaluations for one problem instance.

We first validate the protocol on MaxCut problems. In Fig. 6, we report the average performance over five $n = 12$ graphs with QAOA depth up to 5. The Noiseless (∞) line optimizes the QAOA parameter under the noiseless backend

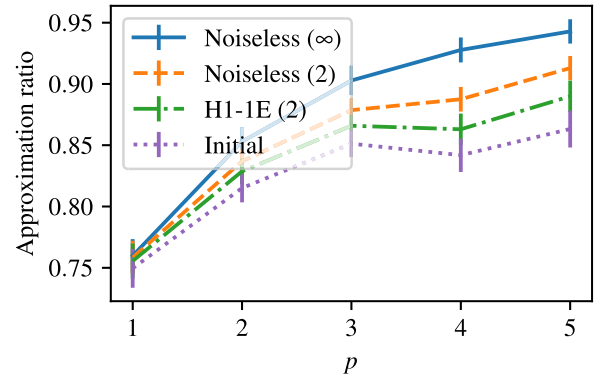


FIG. 6. Optimizing QAOA parameters with $p \in \{1, \dots, 5\}$ for $n = 12$ weighted graphs. Error bars represent the standard error of the mean AR over five instances. The Noiseless (∞) line optimizes the QAOA parameter under the noiseless backend and with an unlimited number of iterations. In other words, it represents the performance of the best achievable parameters. The Noiseless (2) line optimizes the QAOA parameter under the noiseless backend and with two additional iterations, representing the results of the protocol in noiseless simulation. The H1-1E (2) line optimizes with two additional iterations under an emulator backend, which mimics the H1-1 quantum device, representing the deployment of the protocol in a practical scenario. The Initial line presents the performance under the fixed initial parameter.

and with an unlimited number of iterations. In other words, it represents the performance of the best achievable parameters. The Noiseless (2) line optimizes the QAOA parameter under the noiseless backend and with two additional iterations after the first $2p + 1$ initial evaluations, representing the results of the protocol in noiseless simulation. The H1-1E (2) line optimizes with two additional iterations with an emulator backend, which mimics the H1-1 quantum device, representing the deployment of the protocol in a practical scenario. The Initial line presents the performance under the fixed initial parameter. The results validate the effectiveness of the protocol, showing that under different p the protocol can improve the parameter quality nontrivially. At $p = 1$, all the fine-tuning results are close to the initial since the initial parameters are of very high quality. The gap between Noiseless (2) and H1-1E (2) quantifies the impact of the hardware noise.

In Fig. 7, we utilize all the qubits in the H1-1 processor and validate the protocol on three $n = 20$ instances. The hardware results are shown on the H1-1 line. The proposed protocol performs well on quantum hardware, confirming the emulator results. In these selected instances, the initial ARs of $p = 5$ are close to $p = 3$ because the quality of initial parameters is relatively poor for these instances, highlighting the significance of the instance-level fine-tuning. The proposed protocol achieves up to 56.61% relative AR improvement in noiseless simulation while obtaining 46.88% relative AR improvement under hardware noise.

We use our protocol to optimize one $n = 32$ MaxCut instance on the H2-1 processor. The ARs with the standard error over shots are shown in Table I. The “Hardware evaluation” column shows the results directly reported by the hardware optimization. The “Exact evaluation” column shows the results evaluated with a noiseless state vector simulator using

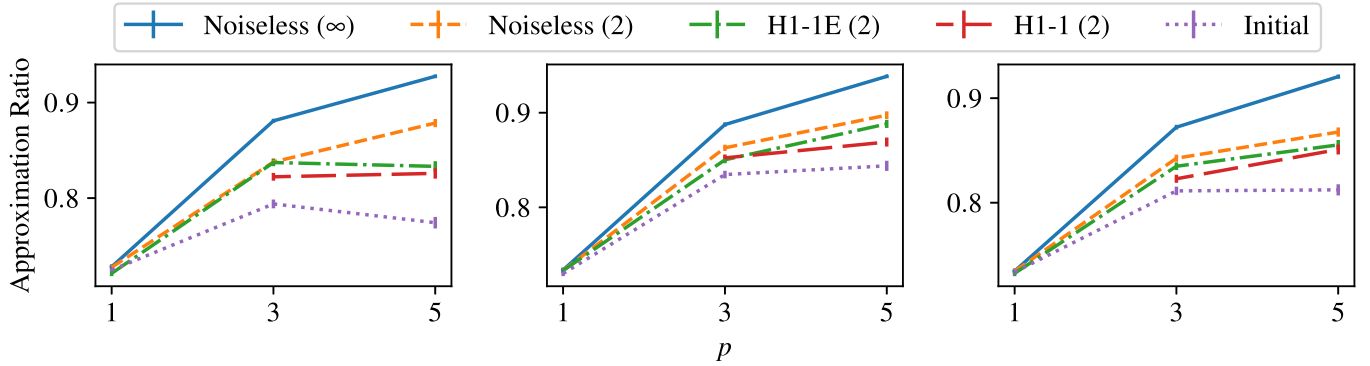


FIG. 7. Optimizing QAOA parameters with $p \in \{1, 3, 5\}$ for three $n = 20$ MaxCut instances. Error bars represent the standard error of the mean AR over limited shots. An error bar is estimated as $\sigma/\sqrt{M}$, where σ is the standard deviation of the approximation ratio associated with the exact QAOA state vector and M is the number of shots. The added H1-1 (2) line optimizes for two additional iterations after the first $2p + 1$ initial evaluations. For the description of other labels, please refer to the caption of Fig. 6.

the parameters found by the hardware optimization. Since the circuit size is larger than in the previous experiments, the gap between the hardware and noiseless simulation also becomes larger. Nonetheless, for $p = 5$, $n = 32$, and a QAOA circuit with 240 two-qubit gates, the protocol is still able to optimize the parameter beyond the high-quality initial parameters. We plot the observed distributions with initial and optimized parameters in Fig. 8.

We also use PO to verify our findings and numerically demonstrate the protocol's general applicability. PO is a harder problem setting because the initial parameter quality is generally poorer than MaxCut's. In addition, the constraint-preserving mixer has a larger overhead in the circuit. We validate the protocol with H1-1E for five $n = 10$ PO instances with p up to 7, as shown in Fig. 9. The QAOA circuit has 97 two-qubit gates for initial state preparation and 65 two-qubit gates for one QAOA layer. We consistently observe the improved AR over the initial parameter setting. Meanwhile, as p becomes larger, the circuit encounters an increasingly greater amount of noise, and the AR performance gap between the noisy and noiseless simulation becomes larger. For the $p = 7$ experiments with 552 two-qubit gates, the relative AR improvement in the noisy simulation is 11.02% while it is 43.52% in the noiseless simulation. Parameter optimization in a highly noisy environment is challenging even under the

fine-tuning setup, suggesting the necessity of deploying error suppression strategies and developing error correction techniques in larger-scale hardware experiments.

IV. DISCUSSION

QAOA has shown algorithmic speedup over the classical state of the art for some problems. High-quality parameters are necessary to realize its quantum speedup. However, although optimal parameters in the average cases are usually available, instance-level fine-tuning is necessary for maximizing the QAOA performance. In this work, we confirm that such fine-tunings can reliably improve the QAOA performance beyond the high-quality initialization with optimization steps linear in the QAOA layer p .

We propose an end-to-end protocol for QAOA parameter setting. Focusing on weighted combinatorial optimization problems within the same family, we assume we have a fixed parameter initialization schedule for the unweighted problems from the same family. We first rescale the weighted problem instance such that the parameters for unweighted problems can be applicable. Then, we need to tune the hyperparameters of the classical optimizer. Under the shot-limited cases,

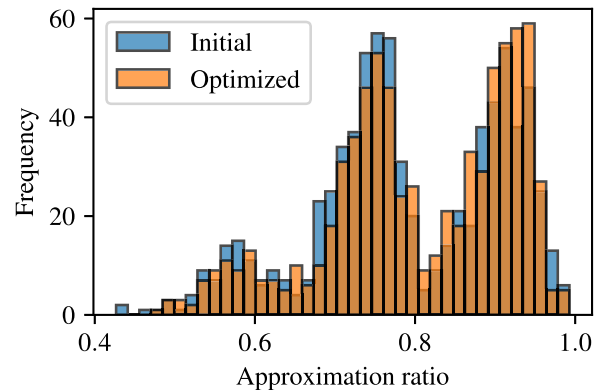


FIG. 8. Shot frequency of solving the $n = 32$ MaxCut with $p = 5$ QAOA in H2-1. The initial bins represent 769 samples from the $p = 5$ QAOA state with initial parameters. The optimized bins represent 769 samples with fine-tuned parameters.

TABLE I. Numerical simulation and hardware demonstration results of $p = 5$ QAOA on an $n = 32$ MaxCut instance. Values in parentheses are the standard error of the mean AR over limited shots. “2 iter” denotes the results, including two iterations after the first $2p + 1$ initial evaluations. The “Hardware evaluation” column shows the results directly reported by the hardware optimization. The “Exact evaluation” column shows the results evaluated with a noiseless state vector simulator using the parameters found by the hardware optimization.

Parameter	Hardware evaluation	Exact evaluation
Initial	0.7963 (0.1367)	0.8084
H2-1 + 2 iter	0.8137 (0.1335)	0.8197
Noiseless + 2 iter	N/A	0.8654

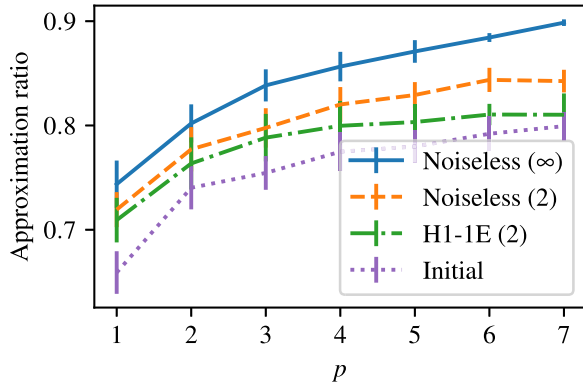


FIG. 9. Optimizing QAOA parameters with $p \in \{1, \dots, 7\}$ for $n = 10$ PO instances. Error bars represent the standard error of the mean AR over five instances. For a description of labels, please refer to the caption of Fig. 6.

we benchmark 12 optimization algorithms and found that COBYLA and BOBYQA perform the best. Considering a realistic shot budget of 10k per problem instance, we use the simplest method, COBYLA. To tune the hyperparameters of COBYLA, we observe that a fixed initial step size works stably in QAOA with different layers for problems of different sizes. Then, we determine the maximum number of iterations of COBYLA to optimize the budget allocation. Under the setting of 10k shots, we assume an equally allocated budget and perform a grid search on the number of shots for one sample. We set the number of steps to be 2 and observe that it consistently works well.

Our protocol and shot budget assumption work for different problem sizes. Figure 10(a) shows the normalized standard deviation of the estimated energy induced by finite sampling for MaxCut of varying problem sizes. Each data point is averaged over 1000 instances. We observe that as the problem size increases, the standard deviation decreases, indicating that fewer shots are needed to maintain the same level of sampling error. We further show the relative AR improvement of our

protocol as a function of the problem size in Fig. 10(b), which confirms that our protocol stays effective at larger problem sizes.

Our protocol can be applied to different problems. We test the protocol with MaxCut and PO instances. For MaxCut, we solve a 32-qubit problem with up to five layers of QAOA and perform hardware demonstrations with both H2-1 and H1-1 devices. For PO, we solve 10-qubit problems with up to seven layers of QAOA and demonstrate in the emulator of the H1-1 device. The protocol is effective and robust to noise. For example, in one instance of 20-qubit MaxCut with $p = 5$, the fine-tuning strategy achieves up to 56.61% relative AR improvement in noiseless simulation while it holds 46.88% relative AR improvement under hardware noise.

This protocol provides an end-to-end solution for QAOA parameter settings on a quantum device. The protocol is also generally applicable when we have many more shots, allowing for more fine-tuning iterations. In such scenarios, several open and interesting directions remain. For example, we can dynamically allocate the shots, allowing each iteration to have a different number of shots. Additionally, we can assign the shot budget to problem instances that perform relatively poorly under the fixed initial parameters. Finally, we remark that our techniques can be combined with other qubit-reuse compilation techniques (e.g., Ref. [63]) to dramatically increase the size of the problem that can be tackled.

V. METHODS

We now present our approach to test our protocol through both numerical simulations and quantum demonstrations. We detail our methods for noiseless and noisy simulations, hardware experiments, and the specific combinatorial optimization problems we considered.

A. Evaluation and data generation

We perform both numerical simulations and quantum demonstrations to derive, tune, and validate our protocol. For

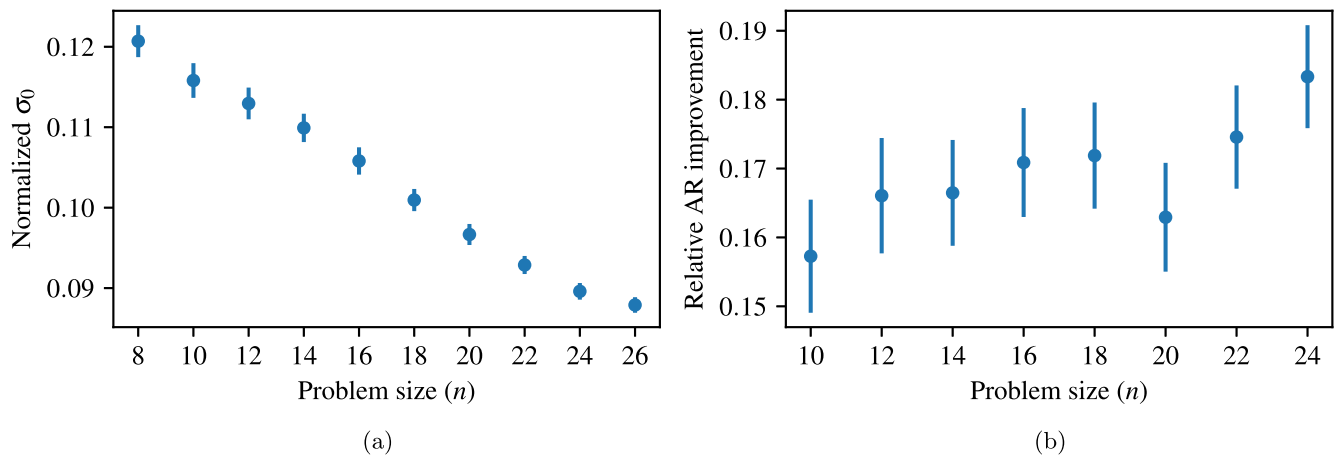


FIG. 10. (a) Normalized standard deviation of the estimated energy (single shot) and (b) relative AR improvement of our protocol (10 000 shots) as a function of problem size n averaged over 1000 $p = 5$ MaxCut instances. Error bars denote the standard error. Considering the absolute change is not substantial, we believe the dip at $n = 20$ in panel (b) is just a normal deviation.

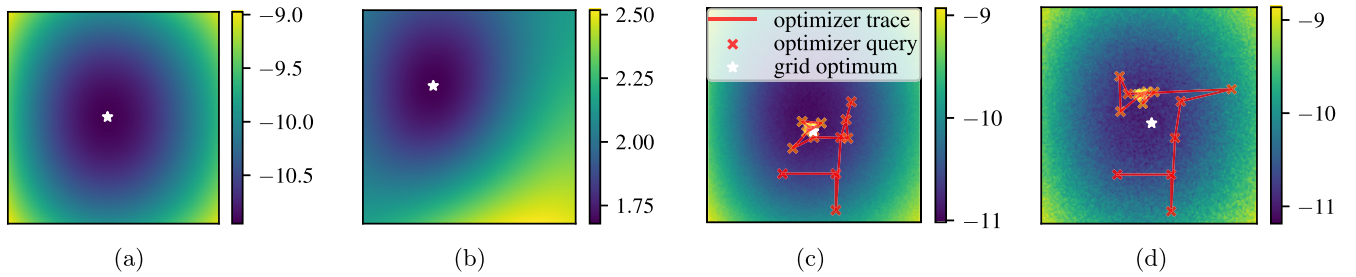


FIG. 11. (a) Energy mean landscape of a $p = 1$ PO instance with a resolution (points along each dimension) of 128 by 128 over the $\pi/4$ by $\pi/4$ region, where the center is the initial point we use in our experiments. (b) Energy standard deviation landscape of the same instance. (c) Sampled landscape showing the effect of 5000 shots per evaluation. An optimization trajectory by COBYLA is overlaid for demonstration. The red to yellow marks represent COBYLA's queries (function evaluations), where the lower left point is the start and the yellow mark is the end. They are connected by a red line ("optimizer trace") to indicate the order. (d) Sampled landscape with 500 shots per evaluation. We see that finite sampling noise has a serious impact on the optimization quality.

noiseless simulations, we employ an optimized MPI-enabled QAOA simulation package, QOKit [67], to simulate up to 32 qubits on two Nvidia A100-80GB GPUs. For noisy simulations, we use the Quantinuum H1 emulator [65]. For hardware experiments, we obtain the results on the Quantinuum H1-1 and H2-1 quantum processors. H1-1 and H2-1 are trapped-ion devices that have full connectivity and high gate fidelity. For detailed specifications of H1-1 and H2-1, please refer to Refs. [65,66]. Whenever the experiment setting is not exact (by exact, we mean noiseless with unlimited shots), we first obtain optimized parameters under the experiment settings and then report the results by exactly evaluating these parameters. This measure is to present the quality of the optimized parameters truthfully and fairly.

We consider two combinatorial optimization problems: weighted maximum cut (MaxCut) and portfolio optimization (PO). For MaxCut, we generate random 3-regular graphs with random edge weights sampled from a mixture of three Gaussian distributions $\{(\mathcal{N}(0, 1), 0.5), (\mathcal{N}(5, 2), 0.3), (\mathcal{N}(10, 1), 0.2)\}$, where 0.5, 0.3, and 0.2 are the probabilities of sampling from the corresponding Gaussian distribution. For PO, we use historical stock market data from Yahoo! Finance as the source for generating the expected return vectors and the covariance matrices. The 60 instances we use for each n are from the first 28 days of each month from January 2015 to December 2019 with n selected stocks from S&P 500 companies that have complete data during that period.

For the optimizer performance comparison, we assume a total budget of 10 000 shots, vary the number of function evaluations from $2p + 2$ to 20 and allocate the shot budget evenly to each evaluation. We also grid search the initial step size for supported optimizers from 0.05 to 0.5 with an increment of 0.05. Further details are described in the next subsection. For initial step size studies, we assume an infinite shot budget. We grid search the initial step size from 0.01 to 0.2 with an increment of 0.01 for MaxCut and from 0.01 to 1 with an increment of approximately 0.05 for PO. For budget allocation studies, we set the number of function evaluations from $2p + 2$ to 50 with an increment of 1 between $2p + 2$ and 20 and an increment of 5 between 20 and 50. The number of shots per evaluation is set using Eq. (17).

B. Low-cost optimizer benchmarking

We employ the quantum optimization helper package OSCAR [12,68] to conduct a computationally tractable evaluation of the various optimizers under different budget allocation strategies. For each QAOA instance, OSCAR computes a discrete energy landscape and then interpolates discrete points on the landscape to answer subsequent function evaluations without actually simulating the circuit. This enables us to quickly evaluate numerous optimization configurations with manageable costs.

Nonetheless, the discrete landscape computation is a grid search, which means its complexity is exponential in the number of points along each dimension. Although OSCAR supports reconstructing a landscape with a small number of sampled points, we choose to evaluate each point on the landscape with exact simulation to avoid approximations. Thus, this approach is viable only for low-depth QAOA landscapes. For our optimizer benchmark experiments, we use $p = 1$ instances and a resolution of 128 by 128 over the $\pi/4$ by $\pi/4$ region where the center is the initial point we use in our experiments. Figure 11(a) shows an example energy landscape we use. Notice that the local optimum (white star in the figure) is very close to the initial point (center of the figure), showing the effectiveness of our initialization strategy.

To simulate the energy with finite sampling errors, in addition to the energy landscapes, we compute the standard deviation landscapes to realize fast energy evaluation with an arbitrary shot.

Figure 11(b) shows an example standard deviation landscape we use. Figures 11(c) and 11(d) show two landscapes sampled from the energy and the standard deviation landscapes with 5000 and 500 shots per evaluation, respectively. We see that the finite sampling adds a noticeable saltlike noise to the landscapes, and fewer shots lead to heavier noise. To demonstrate the impact on optimization, we overlay COBYLA's evaluations on top of the landscape. Note that for demonstration purposes, we start from a lower left point instead of the center, which is the initial point we use in our experiments. We observe that the optimized point (yellow mark in the figure) is seriously affected by the sampling noise.

ACKNOWLEDGMENTS

This material is based upon work supported in part by the U.S. Department of Energy, Office of Science, under Contract No. DE-AC02-06CH11357 and the Office of Science, Office of Advanced Scientific Computing Research, Accelerated Research for Quantum Computing program. The authors thank Danylo Lykov for his help with numerical experiments. T.H., Z.H., R.S., and M.P. thank their colleagues at Global Technology Applied Research of JPMorganChase for their support and helpful discussions.

This paper was prepared for informational purposes with contributions from the Global Technology Applied Research Center of JPMorganChase. This paper is not a product of the Research Department of JPMorganChase or its affiliates. Neither JPMorganChase nor any of its affiliates makes any explicit or implied representation or warranty and none of them accept any liability in connection with this position paper, including, without limitation, with respect to the completeness, accuracy, or reliability of the information contained herein and the potential legal, compliance, tax, or accounting effects thereof. This document is not intended as investment research or investment advice, or as a recommendation, offer, or solicitation for the purchase or sale of any security, financial instrument, financial product or service, or to be used in any way for evaluating the merits of participating in any transaction.

The submitted manuscript includes contributions from UChicago Argonne, LLC, Operator of Argonne National Laboratory (“Argonne”). Argonne, a U.S. Department of Energy Office of Science laboratory, is operated under Contract No. DE-AC02-06CH11357. The U.S. Government retains for itself, and others acting on its behalf, a paid-up nonexclusive, irrevocable worldwide license in said article to reproduce, prepare derivative works, distribute copies to the public, and perform publicly and display publicly, by or on behalf of the Government. The Department of Energy will provide public

access to these results of federally sponsored research in accordance with the DOE Public Access Plan [69].

DATA AVAILABILITY

The code for reproducing the data and figures used in this paper is available in Ref. [70]. The data that support the findings of this article are openly available [71].

APPENDIX: ADDITIONAL RESULTS

Here, we show additional results we have obtained, including the initial step size study for $p \in \{1, 2, 3, 4\}$ (Fig. 12) and the budget allocation study for $p \in \{2, 3, 4, 5\}$ (Figs. 13 and 14).

In Fig. 13, following the similar setup as Fig. 4, we did an additional comparison between COBYLA implemented in NLOpt [54] and PY-BOBYQA [55]. The contours of their performance under different budgets are shown in Fig. 14. PY-BOBYQA is an improved version of BOBYQA implemented in Python. In our numerical experiments, we found that the performance of PY-BOBYQA is similar to the standard BOBYQA implemented in NLOpt [54]. We did not enable the `objfun_has_noise` flag in PY-BOBYQA due to its effect of defaulting to $(1/2)(2p+1)(2p+2)$ initial function evaluations (instead of $4p+1$) and using multirestarts, both of which contradicted our shot-frugal setting. We also wanted to be consistent with the $4p+1$ initial function evaluations we used for BOBYQA in our optimizer comparison experiment (Fig. 2).

Note that for $p = 1$, the initial points are of very high quality and the maximum achievable AR is relatively low due to the shallow QAOA depth. Consequently, the improvable AR is very small, and the optimizer struggles to improve beyond the quality of the initial point, especially with a highly stochastic objective resulting from the low shot budget. Therefore, we do

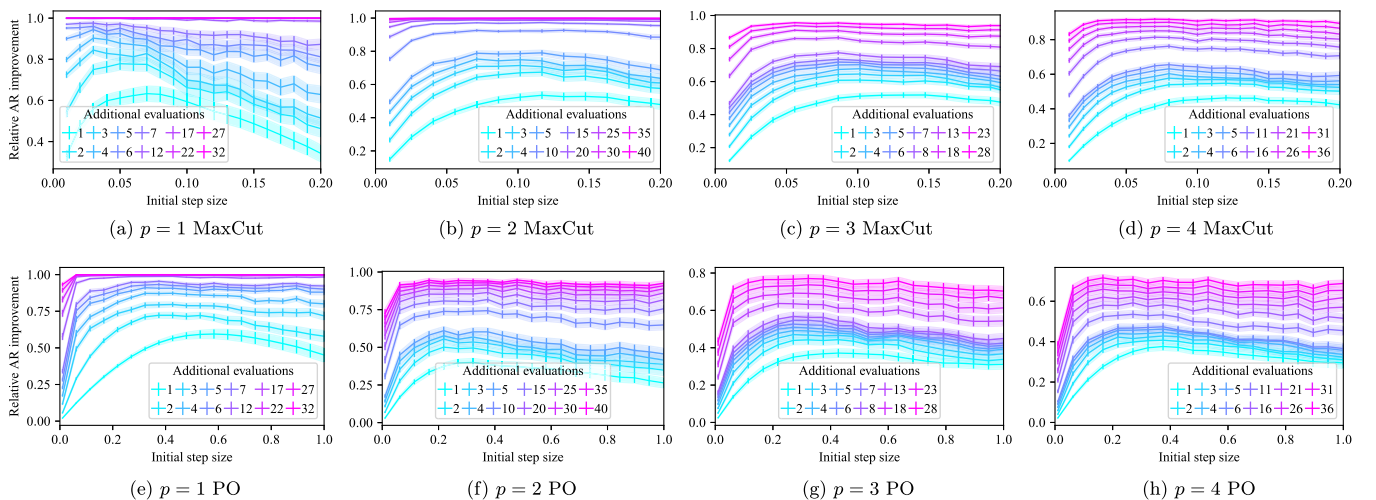


FIG. 12. Mean relative AR improvement (with standard error over instances) of COBYLA on $p \in \{1, 2, 3, 4\}$ MaxCut and PO instances as a function of initial step size. The label of each line represents the number of function evaluations allowed after the initial evaluations. We observe that with a given problem and initialization strategy, COBYLA is generally not sensitive to the initial step size or QAOA depth.

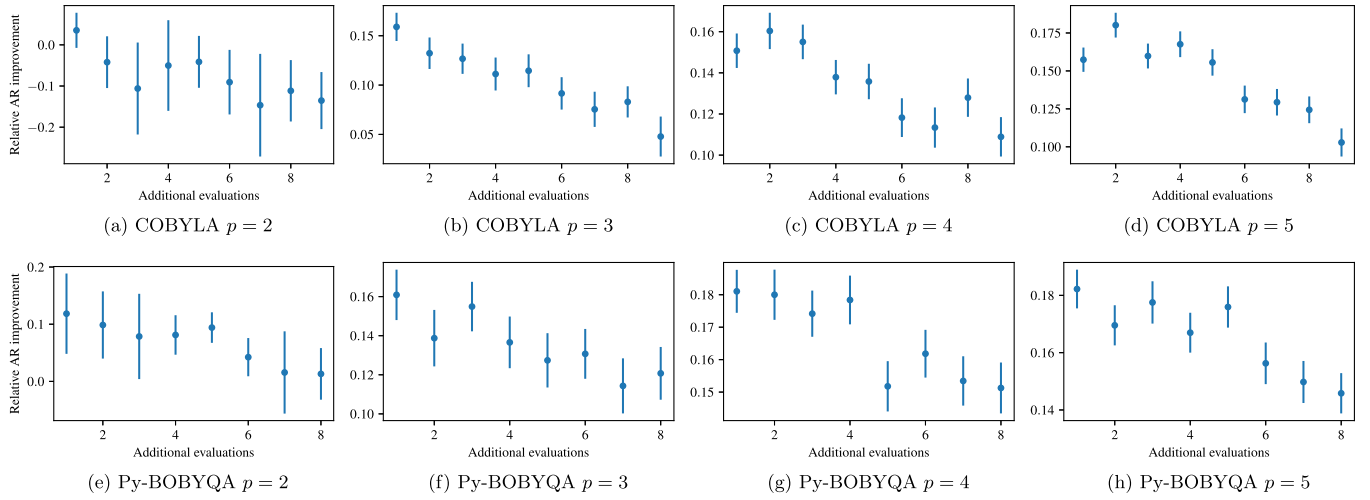


FIG. 13. Mean relative AR improvement (with standard error over instances) of optimizing 1000 $p \in \{2, 3, 4, 5\}$ $n = 12$ MaxCut instances as a function of the number of additional evaluations after the first $2p + 1/4p + 1$ initial evaluations for COBYLA/PY-BOBYQA.

not show $p = 1$ figures or use $p = 1$ MaxCut in the optimizer benchmarking experiments.

For a small p , the difference in the number of initial evaluations does not result in a huge disparity in the number of

shots per evaluation. At $p = 2$, PY-BOBYQA outperforms COBYLA with its quadratic model. For $p \in \{3, 4, 5\}$, their performances are comparable and COBYLA shows a progressive momentum.

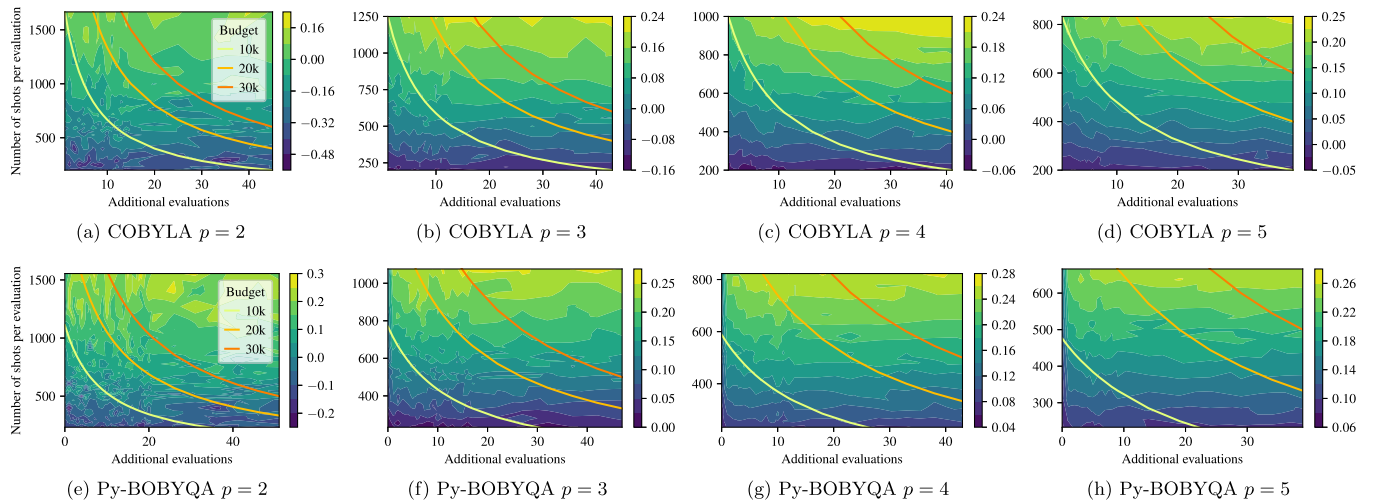


FIG. 14. Contour plot of optimizing 1000 $p \in \{2, 3, 4, 5\}$ $n = 12$ MaxCut instances spanned by the number of additional evaluations and the number of shots per evaluation. The color represents mean relative AR improvement and the three lines correspond to a total budget of 10k, 20k, and 30k, respectively.

- [1] C. Dürr and P. Høyer, A quantum algorithm for finding the minimum, [arXiv:quant-ph/9607014](#).
- [2] A. Montanaro, Quantum-walk speedup of backtracking algorithms, *Theory Comput.* **14**, 1 (2018).
- [3] A. Montanaro, Quantum speedup of branch-and-bound algorithms, *Phys. Rev. Res.* **2**, 013056 (2020).
- [4] R. D. Somma, S. Boixo, H. Barnum, and E. Knill, Quantum simulations of classical annealing processes, *Phys. Rev. Lett.* **101**, 130504 (2008).
- [5] P. Wocjan and A. Abeyesinghe, Speedup via quantum sampling, *Phys. Rev. A* **78**, 042336 (2008).
- [6] T. Hogg and D. Portnov, Quantum optimization, *Inf. Sci.* **128**, 181 (2000).
- [7] E. Farhi, J. Goldstone, and S. Gutmann, A quantum approximate optimization algorithm, [arXiv:1411.4028](#).
- [8] S. Hadfield, Z. Wang, B. O’Gorman, E. G. Rieffel, D. Venturelli, and R. Biswas, From the quantum approximate optimization algorithm to a quantum alternating operator ansatz, *Algorithms* **12**, 34 (2019).
- [9] S. Boulebnane and A. Montanaro, Solving Boolean satisfiability problems with the quantum approximate optimization algorithm, *PRX Quantum* **5**, 030348 (2024).
- [10] R. Shaydulin, C. Li, S. Chakrabarti, M. DeCross, D. Herman, N. Kumar, J. Larson, D. Lykov, P. Minssen, Y. Sun *et al.*, Evidence of scaling advantage for the quantum approximate optimization algorithm on a classically intractable problem, *Sci. Adv.* **10**, eadm6761 (2024).
- [11] Z. He, B. Peng, Y. Alexeev, and Z. Zhang, Distributionally robust variational quantum algorithms with shifted noise, *IEEE Trans. Quantum Eng.* **5**, 3102112 (2024).
- [12] T. Hao, Z. He, R. Shaydulin, M. Pistoia, and S. Tannu, Variational quantum algorithm landscape reconstruction by low-rank tensor completion, in *2024 IEEE International Conference on Quantum Computing and Engineering (QCE), Montreal, QC, Canada* (IEEE, Piscataway, NJ, 2024), Vol. 1, pp. 1184–1190.
- [13] S. H. Sureshbabu, D. Herman, R. Shaydulin, J. Basso, S. Chakrabarti, Y. Sun, and M. Pistoia, Parameter setting in quantum approximate optimization of weighted problems, *Quantum* **8**, 1231 (2024).
- [14] J. Basso, E. Farhi, K. Marwaha, B. Villalonga, and L. Zhou, The quantum approximate optimization algorithm at high depth for MaxCut on large-girth regular graphs and the Sherrington-Kirkpatrick model, in *17th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2022)*, Leibniz International Proceedings in Informatics (LIPIcs), edited by F. Le Gall and T. Morimae (Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2022), Vol. 232, pp. 7:1–7:21.
- [15] J. Wurtz and D. Lykov, The fixed angle conjecture for QAOA on regular MaxCut graphs, [arXiv:2107.00677](#).
- [16] S. Boulebnane and A. Montanaro, Predicting parameters for the quantum approximate optimization algorithm for MaxCut from the infinite-size limit, [arXiv:2110.10685](#).
- [17] R. Shaydulin, P. C. Lotshaw, J. Larson, J. Ostrowski, and T. S. Humble, Parameter transfer for quantum approximate optimization of weighted MaxCut, *ACM Trans. Quantum Comput.* **4**, 1 (2023).
- [18] T. Lubinski, C. Coffrin, C. McGeoch, P. Sathe, J. Apanavicius, and D. E. B. Neira, Optimization applications as quantum performance benchmarks, *ACM Trans. Quantum Comput.* **5**, 1 (2024).
- [19] R. Shaydulin, I. Safro, and J. Larson, Multistart methods for quantum approximate optimization, in *High Performance Extreme Computing Conference* (IEEE, Piscataway, NJ, 2019).
- [20] S. S. Kahani and A. Nobakhti, A novel framework for shot number minimization in quantum variational algorithms, [arXiv:2307.04035](#).
- [21] S. A. Moses, C. H. Baldwin, M. S. Allman, R. Ancona, L. Ascarrunz, C. Barnes, J. Bartolotta, B. Bjork, P. Blanchard, M. Bohn, J. G. Bohnet, N. C. Brown, N. Q. Burdick, W. C. Burton, S. L. Campbell, J. P. Campora, C. Carron, J. Chambers, J. W. Chan, Y. H. Chen *et al.*, A race-track trapped-ion quantum processor, *Phys. Rev. X* **13**, 041052 (2023).
- [22] D. Bluvstein, S. J. Evered, A. A. Geim, S. H. Li, H. Zhou, T. Manovitz, S. Ebadi, M. Cain, M. Kalinowski, D. Hangleiter, J. P. Bonilla Ataides, N. Maskara, I. Cong, X. Gao, P. Sales Rodriguez, T. Karolyshyn, G. Semeghini, M. J. Gullans, M. Greiner, V. Vuletić *et al.*, Logical quantum processor based on reconfigurable atom arrays, *Nature (London)* **626**, 58 (2024).
- [23] M. DeCross, R. Haghshenas, M. Liu, Y. Alexeev, C. H. Baldwin, J. P. Bartolotta, M. Bohn, E. Chertkov, J. Colina, D. DelVento, J. M. Dreiling, C. Foltz, J. P. Gaebler, T. M. Gatterman, C. N. Gilbreth, J. Gray, D. Gresh, N. Hewitt, R. B. Hutson, J. Johansen *et al.*, Computational power of random quantum circuits in arbitrary geometries, *Phys. Rev. X* **15**, 021052 (2025).
- [24] A. Abbas, R. King, H.-Y. Huang, W. J. Huggins, R. Movassagh, D. Gilboa, and J. R. McClean, On quantum backpropagation, information reuse, and cheating measurement collapse, [arXiv:2305.13362](#).
- [25] J. Larson, M. Menickelly, and S. M. Wild, Derivative-free optimization methods, *Acta Numer.* **28**, 287 (2019).
- [26] J. Larson and S. C. Billups, Stochastic derivative-free optimization using a trust region framework, *Comput. Optim. Appl.* **64**, 619 (2016).
- [27] R. Chen, M. Menickelly, and K. Scheinberg, Stochastic optimization using a trust-region method and random models, *Math. Program.* **169**, 447 (2018).
- [28] S. Shashaani, F. S. Hashemi, and R. Pasupathy, ASTRO-DF: A class of adaptive sampling trust-region algorithms for derivative-free stochastic optimization, *SIAM J. Optim.* **28**, 3145 (2018).
- [29] G. Scriva, N. Astrakhantsev, S. Pilati, and G. Mazzola, Challenges of variational quantum optimization with measurement shot noise, *Phys. Rev. A* **109**, 032408 (2024).
- [30] N. Sachdeva, G. S. Harnett, S. Maity, S. Marsh, Y. Wang, A. Winick, R. Dougherty, D. Canuto, Y. Q. Chong, M. Hush *et al.*, Quantum optimization using a 127-qubit gate-model IBM quantum computer can outperform quantum annealers for nontrivial binary optimization problems, [arXiv:2406.01743](#).
- [31] M. Menickelly, Y. Ha, and M. Otten, Latency considerations for stochastic optimizers in variational quantum algorithms, *Quantum* **7**, 949 (2023).
- [32] K. Ito, Latency-aware adaptive shot allocation for run-time efficient variational quantum algorithms, [arXiv:2302.04422](#).
- [33] A. Gu, A. Lowe, P. A. Dub, P. J. Coles, and A. Arrasmith, Adaptive shot allocation for fast convergence in variational quantum algorithms, [arXiv:2108.10434](#).

- [34] A. Arrasmith, L. Cincio, R. D. Somma, and P. J. Coles, Operator sampling for shot-frugal optimization in variational algorithms, [arXiv:2004.06252](#).
- [35] K. J. Sung, J. Yao, M. P. Harrigan, N. C. Rubin, Z. Jiang, L. Lin, R. Babbush, and J. R. McClean, Using models to improve optimizers for variational quantum algorithms, *Quantum Sci. Technol.* **5**, 044008 (2020).
- [36] L. Zhu, S. Liang, C. Yang, and X. Li, Optimizing shot assignment in variational quantum eigensolver measurement, *J. Chem. Theory Comput.* **20**, 2390 (2024).
- [37] C. Moussa, M. H. Gordon, M. Baczyk, M. Cerezo, L. Cincio, and P. J. Coles, Resource frugal optimizer for quantum machine learning, *Quantum Sci. Technol.* **8**, 045019 (2023).
- [38] J. M. Kübler, A. Arrasmith, L. Cincio, and P. J. Coles, An adaptive optimizer for measurement-frugal variational algorithms, *Quantum* **4**, 263 (2020).
- [39] A. M. Polloreno and G. Smith, The QAOA with slow measurements, [arXiv:2205.06845](#).
- [40] L. Cheng, Y.-Q. Chen, S.-X. Zhang, and S. Zhang, Quantum approximate optimization via learning-based adaptive optimization, *Commun. Phys.* **7**, 83 (2024).
- [41] V. Akshay, D. Rabinovich, E. Campos, and J. Biamonte, Parameter concentrations in quantum approximate optimization, *Phys. Rev. A* **104**, L010401 (2021).
- [42] P. Billingsley, *Probability and Measure* (John Wiley & Sons, 2017).
- [43] J. Larson, M. Menickelly, and J. Shi, A novel noise-aware classical optimizer for variational quantum algorithms, *INFORMS J. Comput.* **37**, 1 (2025).
- [44] A. Bärtshi and S. Eidenbenz, Deterministic preparation of Dicke states, in *International Symposium on Fundamentals of Computation Theory* (Springer, 2019), pp. 126–139.
- [45] M. J. D. Powell, A direct search optimization method that models the objective and constraint functions by linear interpolation, in *Advances in Optimization and Numerical Analysis*, Mathematics and Its Applications, edited by S. Gomez and J.-P. Hennart (Springer, 1994), Vol. 275, pp. 51–67.
- [46] M. J. D. Powell, *The BOBYQA algorithm for bound constrained optimization without derivatives*, Tech. Rep. NA2009/06 (Department of Applied Mathematics and Theoretical Physics, Cambridge University, Cambridge, 2009).
- [47] J. A. Nelder and R. Mead, A simplex method for function minimization, *Comput. J.* **7**, 308 (1965).
- [48] C. H. da Silva Santos, M. S. Goncalves, and H. E. Hernandez-Figueroa, Designing novel photonic devices by bio-inspired computing, *IEEE Photonics Technol. Lett.* **22**, 1177 (2010).
- [49] J. M. Gablonsky and C. T. Kelley, A locally-biased form of the DIRECT algorithm, *J. Global Optim.* **21**, 27 (2001).
- [50] P. Kaelo and M. Ali, Some variants of the controlled random search algorithm for global optimization, *J. Optim. Theory Appl.* **130**, 253 (2006).
- [51] J. C. Spall, Multivariate stochastic approximation using a simultaneous perturbation gradient approximation, *IEEE Trans. Autom. Control* **37**, 332 (1992).
- [52] A. S. Berahas, L. Cao, K. Choromanski, and K. Scheinberg, A theoretical and empirical comparison of gradient approximations in derivative-free optimization, *Found. Comput. Math.* **22**, 507 (2022).
- [53] C. T. Kelley, *Implicit Filtering* (SIAM, 2011).
- [54] S. G. Johnson, The NLOpt nonlinear-optimization package, 2007, <https://github.com/stevengj/nlopt>.
- [55] C. Cartis, J. Fiala, B. Marteau, and L. Roberts, Improving the flexibility and robustness of model-based derivative-free optimization solvers, *ACM Trans. Math. Softw.* **45**, 1 (2019).
- [56] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey *et al.*, SciPy 1.0: Fundamental algorithms for scientific computing in Python, *Nat. Methods* **17**, 261 (2020).
- [57] T. M. Ragonneau and Z. Zhang, PDFO: A cross-platform package for Powell's derivative-free optimization solvers, [arXiv:2302.13246](#).
- [58] W. Lavrijsen, A. Tudor, J. Müller, C. Iancu, and W. De Jong, Classical optimizers for noisy intermediate-scale quantum devices, in *International Conference on Quantum Computing and Engineering* (IEEE, 2020), pp. 267–277.
- [59] M. J. Powell, UOBYQA: Unconstrained optimization by quadratic approximation, *Math. Program.* **92**, 555 (2002).
- [60] M. J. D. Powell, The NEWUOA software for unconstrained optimization without derivatives, in *Large-Scale Nonlinear Optimization*, Nonconvex Optimization and Its Applications, edited by G. D. Pillo and M. Roma (Springer, 2006), Vol. 83, pp. 255–297.
- [61] M. J. D. Powell, On fast trust region methods for quadratic models with linear constraints, *Math. Program. Comput.* **7**, 237 (2015).
- [62] W. Huyer and A. Neumaier, SNOBFIT—Stable noisy optimization by branch and fit, *ACM Trans. Math. Software* **35**, 1 (2008).
- [63] M. DeCross, E. Chertkov, M. Kohagen, and M. Foss-Feig, Qubit-reuse compilation with mid-circuit measurement and reset, *Phys. Rev. X* **13**, 041057 (2023).
- [64] M. P. Harrigan, K. J. Sung, M. Neeley, K. J. Satzinger, F. Arute, K. Arya, J. Atalaya, J. C. Bardin, R. Barends, S. Boixo *et al.*, Quantum approximate optimization of non-planar graph problems on a planar superconducting processor, *Nat. Phys.* **17**, 332 (2021).
- [65] Quantinuum H1-1, 2023, <https://www.quantinuum.com/>.
- [66] Quantinuum H2-1, 2024, <https://www.quantinuum.com/>.
- [67] D. Lykov, R. Shaydulin, Y. Sun, Y. Alexeev, and M. Pistoia, Fast simulation of high-depth QAOA circuits, in *Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis, SC-W 2023* (ACM, 2023).
- [68] T. Hao, K. Liu, and S. Tannu, Enabling high performance debugging for variational quantum algorithms using compressed sensing, in *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Association for Computing Machinery, New York, 2023).
- [69] <http://energy.gov/downloads/doe-public-access-plan>.
- [70] https://github.com/jpmorganchase/End-to-End_Protocol_for_High-Quality_QAOA_Parameters.
- [71] T. Hao, Z. He, R. Shaydulin, J. Larson, and M. Pistoia, Data from “End-to-end protocol for high-quality QAOA parameters with few shots” [Data set], Zenodo (2024), <https://doi.org/10.5281/zenodo.12209739>.